

Advanced Programming in Python
Project: DNA Sequence Assembly
Maryam Shojaei Shahrokhbabadi()

Overview

The project reconstructs a DNA sequence from short fragments by cleaning input data, generating sequences, and building a de Bruijn graph to represent overlaps. It then reconstructs the original DNA using an Eulerian path and saves both the sequence and a visual graph. The code is modular, with each major step—file parsing, data cleaning, sequence generation, graph construction and visualization, sequence assembly, and output saving—handled in separate functions to enhance readability, reusability, and testability.

Implementation Details

First, the code extracts parameters from the input filename `'DNA_x_k'` using a regular expression, capturing the segment number (`x`) and the `k`-mer size (`k`). If the filename matches the expected format, it sets the output filename (`out_file`) as `'DNA_x'` and assigns `k` as an integer; otherwise, it prints an error message.

To run the code, assign the name of the CSV file (without extension) as a string to the variable `file_name`. The corresponding CSV file must be located inside a folder named `csv-files`, which is in the same directory as the script.

Once the filename is validated and parsed, the DNA sequence reconstruction process begins:

- Read the data using `read_csv(file_name)` function that get a string `'DNA_x_k'` as an input.
- Data cleaning through `clean_data(a)` function.
- Generate JSON sequences from the dataframe using `generate_sequences(data)` function.
- Construct de Bruijn graph by `construct_graph(JSON_sequences, k)` function. `K` is extracted from the input filename at first step.

Note: In `project_test.py`, the expected output of the `construct_graph` function is a list of edges rather than a graph object. Therefore, I modified `construct_graph` to return a list. However, after this step, I converted the list back into a graph object (`debruijn_graph`) to use it as input for the `plot_graph` and `construct_dna_sequence` functions.

- plot the constructed graph by `plot_graph(debruijn_graph, out_file)` function. `out_file` is extracted from the input filename at first step.
- Construct the DNA sequence using the `construct_dna_sequence(graph)` function. It uses `debruijn_graph` object as an input. The check for whether the de Bruijn graph can be used to reconstruct a sequence is performed within this function.

- Save the DNA sequence or write the error message using `save_output(string, out_file)`.

More details for each step is as followed:

1- read_csv(name)

This function reads a CSV file containing DNA segment data from the folder named csv-files, located in the same directory as the script. Its input (`name`) is a string (e.g., 'DNA_1_5'), and it returns a DataFrame. The full filename is built by appending `.csv`. The file is read using `pandas.read_csv` with `header=None` to treat the first row as data (not column names). Custom column names (`SegmentNr`, `Position`, `A`, `C`, `G`, `T`) are assigned. Pandas is chosen for its reliability and efficiency in managing tabular data.

2- clean_data(data)

This function cleans the input DataFrame by removing invalid or inconsistent DNA segments through four steps. Initially, the plan was to validate each segment in one go, but this led to complex nested conditionals. Instead, the code processes the data fully at each cleaning step, making it easier to implement but potentially less optimal. After applying the four helper functions, the final output of `clean_data` is a DataFrame representing a cleaned version of the input. Handling duplicated positions and segments first reduces unnecessary checks in later validations. Although this may involve some redundant computations, it keeps the logic simple and modular. Moreover, for each step if the data frame is empty an error 'dataframe is empty' is raised. The steps include:

deal_Duplicated_position(data): This function removes segments with duplicated positions. It loops through each unique `SegmentNr`, sorts by `Position`, and uses a NumPy array for efficient row access. It tracks seen positions in a dictionary: if a position repeats with identical nucleotide values, the duplicate is marked for removal; if the values differ, the entire segment is flagged and removed. After processing, only clean duplicates are dropped unless a conflict is found, in which case the full segment is discarded. The cleaned DataFrame is returned for further processing.

deal_Missing_position(data): This function removes segments that have missing position values. It loops through each unique segment, identifies all expected position values from 1 to the segment's maximum position, and counts how many of these exist in the data. If the count of existing positions is less than the maximum expected, the whole segment is considered invalid. The indices of all rows in that segment are collected using `indexes_drop = data[index].index`, and those rows are removed from the DataFrame using `data.drop(indexes_drop)`. After checking all segments, the cleaned DataFrame is returned as an output for next step.

deal_Wrong_position(data): This function removes segments with invalid rows based on nucleotide weights. It loops through each unique segment and sorts the rows by position for consistency. The rows are converted to a NumPy array for faster processing. For each row, it calculates the sum of the nucleotide values (A, C, G, T). If the sum is not equal to 1 (indicating either all zeros or more than one value is equal to one), the segment is marked as invalid. If any invalid row is found, the entire segment is dropped. Finally, the cleaned DataFrame is returned.

deal_Duplicated_segments(data): This function efficiently removes duplicate segments by storing `seen_sequences` in a dictionary for quick lookup. It processes each segment by sorting its rows, converting them into a tuple-based sequence (excluding `SegmentNr`), and checks if it has already been recorded. If so, the segment is marked for removal. This avoids repeated comparisons and ensures only unique segments remain in the DataFrame.

3- generate_sequences(data)

This function generates a JSON object containing DNA sequences for each segment in the cleaned DataFrame. The output is a dictionary with segment numbers as keys and their corresponding json version of the sequences as values. Because dictionary provides efficient data retrieval and organization. The function groups the data by segment and uses a helper function, `decode(segment)`, to convert each group into a sequence.

decode(segment): This function reconstructs a DNA sequence from a single segment by sorting rows based on position and identifying the nucleotide (A, C, G, or T) marked with a value of 1 in each row. It builds the sequence in order, ensuring accurate representation of the segment's nucleotide structure.

SegmentNr	Position	A	C	G	T	Json sequence
1	1	1	0	0	0	{1: 'ATC'}
1	2	0	0	0	1	
1	3	0	1	0	0	

4- construct_graph(JSON_sequences, k)

This function constructs a de Bruijn graph from DNA sequences using k-mers. It takes a dictionary of sequences (`json_data`) and a k-mer length `k` (must be >2). It first generates k-mers with `kmers()`, then splits each into $(k-1)$ -length left and right parts using `L_R_segments()`. Each L-R pair is added as a directed edge in a `MultiDiGraph`, allowing repeated transitions. Although the project specifies returning a graph object, this version returns a list of edges to match the test file requirements.

kmers(JSON_sequences, k): This function generates all k-mers (substrings of length `k`) for each DNA sequence in the input dictionary (`k` must be $\leq$ length of each sequence). It loops through each segment's sequence, sliding a window of size `k` across the string to extract all overlapping `k`-length substrings. These k-mers are stored in a dictionary called `kmer_dict`, where each key corresponds to a segment number and each value is the list of k-mers extracted from that segment's sequence.

L_R_segments(kmer_dict, k): This function receives the k-mer dictionary and value of `k` as inputs, and returns a list containing all $(k-1)$ -length L and R segments from each k-mer. These will be used as graph nodes.

Note: in the main structure, the output of `construct_graph(JSON_sequences, k)`, which is a list, is converted to a graph object for further process.

5- plot_graph(graph, out_file)

The function visualizes a graph and saves it as a PNG image. It arranges nodes in a grid structure with 5 nodes per row for clarity. Instead of using edge curvature, it employs `edge_labels` to indicate the number of repeated edges between nodes, utilizing `count_parallel_edges(graph)` to gather this information. The final visualization is saved with the specified filename.

count_parallel_edges(graph): The function processes a directed multigraph and returns a dictionary that counts the occurrences of each edge between pairs of nodes. It iterates through all edges, including their keys, and stores the count of each (u, v) edge, allowing for the identification of repeated transitions between the same nodes.

6- construct_dna_sequence(graph)

This function is responsible for reconstructing the original DNA sequence from the de Bruijn graph. It gives a graph as an Input and returns a string which represent either the DNA sequence or the Error message. It first checks whether the graph is valid for constructing a Eulerian path using the `is_valid_graph` function. If the graph is valid, the function determines the appropriate start node with `startnode_eulerian_path` and then finds a Eulerian path through the graph using `find_eulerian_path`. The function then reconstructs the DNA sequence by concatenating the first character of each node in the Eulerian path, adding the entire last node at the end to complete the sequence. If the graph does not meet the criteria for a Eulerian path, it returns a message 'that the DNA sequence cannot be constructed'. The function also prints the Eulerian path by joining the names of all nodes in the path with a " - " separator. This is done using the line `for_print = ' - '.join(Euler_path)`.

is_valid_graph: The function checks if a directed graph can have a Eulerian path, returning True or False. It first verifies the graph's connectivity using `check_connected(graph)`. Then, it compares the in-degree and out-degree of each node, counting nodes with unequal degrees and storing the differences as tuples in a list. If no nodes have unequal in-degree and out-degree, the function returns True. If exactly two nodes have differing degrees, it extracts the degree differences using `diff_vector = [diff for _, diff in nodes]`, which creates a list of the degree difference while ignoring node names. The function then checks whether `diff_vector` equals `[-1, 1]`, which confirms that one node has one extra outgoing edge and the other has one extra incoming edge. If this condition is met, it returns True. In all other cases, or if the graph is not connected, it returns False.

check_connected(graph): The function converts the input graph to an undirected version and uses depth-first search (DFS) to explore all reachable nodes from the first node. It tracks visited nodes and continues until no nodes remain to visit. If all nodes are visited, it returns True, indicating the graph is connected; otherwise, it returns False.

startnode_eulerian_path(graph): This function returns the starting node for a Eulerian path. It first looks for a node with one more outgoing edge than incoming. If none is found, it returns the first node in sorted order as a default.

find_eulerian_path(graph, start_node): The function is responsible for constructing a complete Eulerian path from a given directed multigraph and a starting node. It works by breaking down the task into manageable steps using helper functions. Since this function relies on multiple helpers, the overall approach and logic will be discussed in a separate section for clarity.

7- find_eulerian_path(graph, start_node)

This function uses different helper functions to construct Eulerian path from a given directed multigraph and a starting node.

out_edges_multidigraph(graph): This function converts the graph's edge structure into a dictionary to handle multiple parallel edges more effectively. It iterates through all the edges in the directed multigraph and constructs a dictionary called `out_edges`, where each key represents a source node and the corresponding value is a list of destination nodes it has outgoing edges to.

path_to_stuck(start_node, out_edges): This function follows the chain of available outgoing edges, moving from one node to the next by always selecting the first available outgoing edge. It appends each visited node to the `path` and removes the used edges from the `out_edges` dictionary to prevent re-visiting them. The traversal continues until it reaches a node with no remaining outgoing edges. Within the `find_eulerian_path(graph, start_node)` function, it is specifically used to generate an initial path and update the `out_edges` structure accordingly.

extend_euler_path(): At the third step of the `find_eulerian_path(graph, start_node)` function, the `extend_euler_path()` function is used to examine the current path and look for nodes within it that still have unused outgoing edges. If such a node is found, a new path is created from that node using `path_to_stuck()`, and the new path is spliced into the original path at the correct position. This process is repeated **recursively** until no unused edges remain, ensuring that every edge is visited exactly once, as required in the Eulerian path.

8- save_output(string, out_file)

This function writes a given string `s` to a text file named `filename.txt`. It uses a `with open(...)` block to safely open the file in write mode, ensuring the file is properly closed afterward. If the file already exists, its contents will be overwritten.

Evaluation

I have successfully completed all 8 tasks of the project. The implementation correctly constructs the de Bruijn graph, checks for a Eulerian path, and reconstructs the DNA sequence accordingly. In this case, there is only one valid Eulerian path, so only one sequence can be constructed. No bonus tasks were included. However, one test related to data cleaning failed when the input DataFrame was empty.

Challenges

One of the main challenges I faced was plotting the graph clearly and informatively. Despite my efforts to arrange nodes in an aligned layout, the final figure still suffers from overlapping and crossing edges, which reduces its visual clarity. Additionally, I considered reporting the number of segments and positions removed during the data cleaning process, but since this was not requested, I avoided adding it to prevent potential conflicts in running the code.

Tests Description

For 5 main functions, at least 3 test cases are designed as followed:

For **test_clean_data**, four test cases are designed. Both input and output are Pandas DataFrames.

Test 1: It is for Wrong position test. Segment 1 has invalid record (more than one 1) and should be removed

Test 2: Segment 1 is missing a position (position 4 is absent); this incomplete segment should be removed.

Test 3: Segment 1 has duplicated positions with conflicting nucleotide records; the entire segment should be deleted due to inconsistency.

Test 4: There are duplicated positions within segment 1 and a completely duplicated segment 3 (identical to segment 1). Segment 3 should be removed entirely, and one of the duplicated positions in segment 1 should be deleted. (This test failed, expected output is empty DataFrames).

To test **generate_sequences**, I converted nucleotide indicators into a DNA string for a single segment, ensured correct sequence generation for multiple segments, and used minimal input to verify functionality with small datasets, confirming accurate DNA sequence construction from cleaned DataFrame inputs.

To test **construct_graph**, I used small segments and varied the k values. For example, json_data for test 3 is {5: 'GGA'}, and k=2. Then 2-mers for this segments are: GG, GA and the following L and R of each one of them: (GG: G, G and GA: G, A), so the corresponding list for graphs is [('G', 'G'), ('G', 'A')]

To test **is_valid_graph**, I defined several edge lists representing both valid and invalid graphs. For example, [('A', 'B'), ('A', 'C'), ('D', 'E')] is a non-connected graph and should return False. In another case, [('A', 'B'), ('B', 'C'), ('C', 'A'), ('A', 'C'), ('C', 'B'), ('A', 'B')], node B has 3 outgoing edges and 1 incoming edge, while nodes A and C have equal in and out degrees, making the graph invalid due to the degree imbalance.

To test **construct_dna_sequence**, I defined several edge lists representing both valid and invalid graphs. For example, the list [('AAA', 'AAC'), ('AAC', 'ACA'), ('ACA', 'CAC')] forms a valid Eulerian path: AAA → AAC → ACA → CAC, which corresponds to the reconstructed DNA sequence "AAACAC".

Example Run

1. Read the csv file given its name.

I used the file *DNA_2_5.csv*. The first step involved reading the data using *read_csv()*, which returned a DataFrame with 91 rows, as shown below:

```
   SegmentNr  Position  A  C  G  T
0           1         1  0  0  0  1
1           1         2  0  0  0  1
2           1         2  0  0  0  1
3           1         3  1  0  0  0
4           1         4  1  0  0  0
..          ...      ...  ..  ..  ..
86          12         5  1  0  0  0
87          12         6  0  1  0  0
88          13         4  0  1  0  0
89          13         1  1  0  0  0
90          13         2  0  1  0  0

[91 rows x 6 columns]
```

2. Clean the data of the dataframe.

Then, *clean_data()* removes invalid entries and prepared the dataset for sequence generation. This function performs data cleaning in four steps:

```
step1 = deal_Duplicated_position(data)
```

After this step, 4 rows were deleted: row 2, due to Duplicated position in a segment

Before step 1

```
   SegmentNr  Position  A  C  G  T
0           1         1  0  0  0  1
1           1         2  0  0  0  1
2           1         2  0  0  0  1
3           1         3  1  0  0  0
4           1         4  1  0  0  0
..          ...      ...  ..  ..  ..
86          12         5  1  0  0  0
87          12         6  0  1  0  0
88          13         4  0  1  0  0
89          13         1  1  0  0  0
90          13         2  0  1  0  0

[91 rows x 6 columns]
```

After step1

```
   SegmentNr  Position  A  C  G  T
0           1         1  0  0  0  1
1           1         2  0  0  0  1
3           1         3  1  0  0  0
4           1         4  1  0  0  0
5           1         5  0  0  0  1
..          ...      ...  ..  ..  ..
86          12         5  1  0  0  0
87          12         6  0  1  0  0
88          13         4  0  1  0  0
89          13         1  1  0  0  0
90          13         2  0  1  0  0

[87 rows x 6 columns]
```

And also whole Segment 11 was deleted because it contained duplicated positions with differing records.

Before step 1

	SegmentNr	Position	A	C	G	T
75	9	3	0	1	0	0
76	10	3	0	0	0	0
77	10	1	1	0	0	0
78	10	2	0	1	0	0
79	11	1	0	1	0	0
80	11	1	1	0	0	0
81	11	2	0	1	0	0
82	12	1	1	0	0	0
83	12	2	0	1	0	0
84	12	3	0	0	0	1
85	12	4	0	1	0	0

After step1

	SegmentNr	Position	A	C	G	T
76	10	3	0	0	0	0
77	10	1	1	0	0	0
78	10	2	0	1	0	0
82	12	1	1	0	0	0
83	12	2	0	1	0	0
84	12	3	0	0	0	1
85	12	4	0	1	0	0
86	12	5	1	0	0	0
87	12	6	0	1	0	0
88	13	4	0	1	0	0
89	13	1	1	0	0	0

```
step2 = deal_Duplicated_segments(step1)
```

The entire Segment 12 was removed after step 2, because it was a duplicate of Segment 5.

Before step 2

	SegmentNr	Position	A	C	G	T
0	1	1	0	0	0	1
1	1	2	0	0	0	1
3	1	3	1	0	0	0
4	1	4	1	0	0	0
5	1	5	0	0	0	1
..	...	...	..	..	..	..
86	12	5	1	0	0	0
87	12	6	0	1	0	0
88	13	4	0	1	0	0
89	13	1	1	0	0	0
90	13	2	0	1	0	0

[87 rows x 6 columns]

After step2

	SegmentNr	Position	A	C	G	T
0	1	1	0	0	0	1
1	1	2	0	0	0	1
3	1	3	1	0	0	0
4	1	4	1	0	0	0
5	1	5	0	0	0	1
..	...	...	..	..	..	..
77	10	1	1	0	0	0
78	10	2	0	1	0	0
88	13	4	0	1	0	0
89	13	1	1	0	0	0
90	13	2	0	1	0	0

[81 rows x 6 columns]

```
step3 = deal_Wrong_position(step2)
```

After going through step3, whole segments 9 and 10 were deleted. Because both of them had wrong positions.

Before step 3

	SegmentNr	Position	A	C	G	T
0	1	1	0	0	0	1
1	1	2	0	0	0	1
3	1	3	1	0	0	0
4	1	4	1	0	0	0
5	1	5	0	0	0	1
..	...	...	..	..	..	..
77	10	1	1	0	0	0
78	10	2	0	1	0	0
88	13	4	0	1	0	0
89	13	1	1	0	0	0
90	13	2	0	1	0	0

[81 rows x 6 columns]

After step3

	SegmentNr	Position	A	C	G	T
0	1	1	0	0	0	1
1	1	2	0	0	0	1
0	1	1	0	0	0	1
1	1	2	0	0	0	1
3	1	3	1	0	0	0
4	1	4	1	0	0	0
5	1	5	0	0	0	1
..	...	...	..	..	..	..
71	8	11	0	0	0	1
72	8	12	0	0	1	0
88	13	4	0	1	0	0
89	13	1	1	0	0	0
90	13	2	0	1	0	0

[75 rows x 6 columns]

```
final_data = deal_Missing_position(step3)
```

After last step, Segment 13 was removed due to missing positions in the segment.

After completing all cleaning steps, the final DataFrame contains 72 rows, as shown below:

	SegmentNr	Position	A	C	G	T
0	1	1	0	0	0	1
1	1	2	0	0	0	1
3	1	3	1	0	0	0
4	1	4	1	0	0	0
5	1	5	0	0	0	1
..	...	...	..	..	..	..
68	8	8	0	1	0	0
69	8	9	1	0	0	0
70	8	10	0	1	0	0
71	8	11	0	0	0	1
72	8	12	0	0	1	0

[72 rows x 6 columns]

3. Generate JSON sequences from the dataframe.

The cleaned data is then used as input for the function:

```
JSON_sequences = generate_sequences(data)
```

This function returns a dictionary containing the JSON sequence for each segment.

The first 10 rows of the DataFrame correspond to the first segment, whose JSON sequence is 'TTAATTA', as shown below:

	SegmentNr	Position	A	C	G	T
0	1	1	0	0	0	1
1	1	2	0	0	0	1
3	1	3	1	0	0	0
4	1	4	1	0	0	0
5	1	5	0	0	0	1
6	1	6	0	0	0	1
7	1	7	1	0	0	0
8	2	1	0	1	0	0
9	2	2	0	0	0	1
10	2	3	1	0	0	0
None						
{1: 'TTAATTA', 2: 'CTAATTACTCACTGG', 3: 'ATTACTC', 4: 'CTGGGT', 5: 'ACTCAC', 6: 'GGGTCAC', 7: 'TCACTGGCTAA', 8: 'CACTACGCACTG'}						

4. Construct de Bruijn graph.

The function *kmers(JSON_sequences, k)* gives *JSON_sequences* from last step and kmer value k and the returns k-mers for each segment:

```
{1: ['TTAAT', 'TAATT', 'AATTA'],
2: ['CTAAT', 'TAATT', 'AATTA', 'ATTAC', 'TTACT', 'TACTC', 'ACTCA', 'CTCAC', 'TCACT',
'CACTG', 'ACTGG'],
3: ['ATTAC', 'TTACT', 'TACTC'],
4: ['CTGGG', 'TGGGT'],
5: ['ACTCA', 'CTCAC'],
6: ['GGGTC', 'GGTCA', 'GTCAC', 'TCACT'],
7: ['TCACT', 'CACTG'GGCT', 'GGCTA', 'GCTAA', 'ACTGG', 'CTGGC', 'TGGCT', 'GGCTA',
'GCTAA'],
8: ['CACTA', 'ACTAC', 'CTACG', 'TACGC', 'ACGCA', 'CGCAC', 'GCACT', 'CACTG']}
```

And then the output of *L_R_segments(kmer_dict, k)* is a List as followed:

```
['TTAA', 'TAAT', 'TAAT', 'AATT', 'AATT', 'ATTA', 'CTAA', 'TAAT', 'TAAT', 'AATT', 'AATT',
'ATTA', 'ATTA', 'TTAC', 'TTAC', 'TACT', 'TACT', 'ACTC', 'ACTC', 'CTCA', 'CTCA', 'TCAC',
'TCAC', 'CACT', 'CACT', 'ACTG', 'ACTG', 'CTGG', 'ATTA', 'TTAC', 'TTAC', 'TACT', 'TACT',
'ACTC', 'CTGG', 'TGGG', 'TGGG', 'GGGT', 'ACTC', 'CTCA', 'CTCA', 'TCAC', 'GGGT', 'GGTC',
'GGTC', 'GTCA', 'GTCA', 'TCAC', 'TCAC', 'CACT', 'TCAC', 'CACT', 'CACT', 'ACTG', 'ACTG',
'CTGG', 'CTGG', 'TGGC', 'TGGC', 'GGCT', 'GGCT', 'GCTA', 'GCTA', 'CTAA', 'CACT', 'ACTA',
```

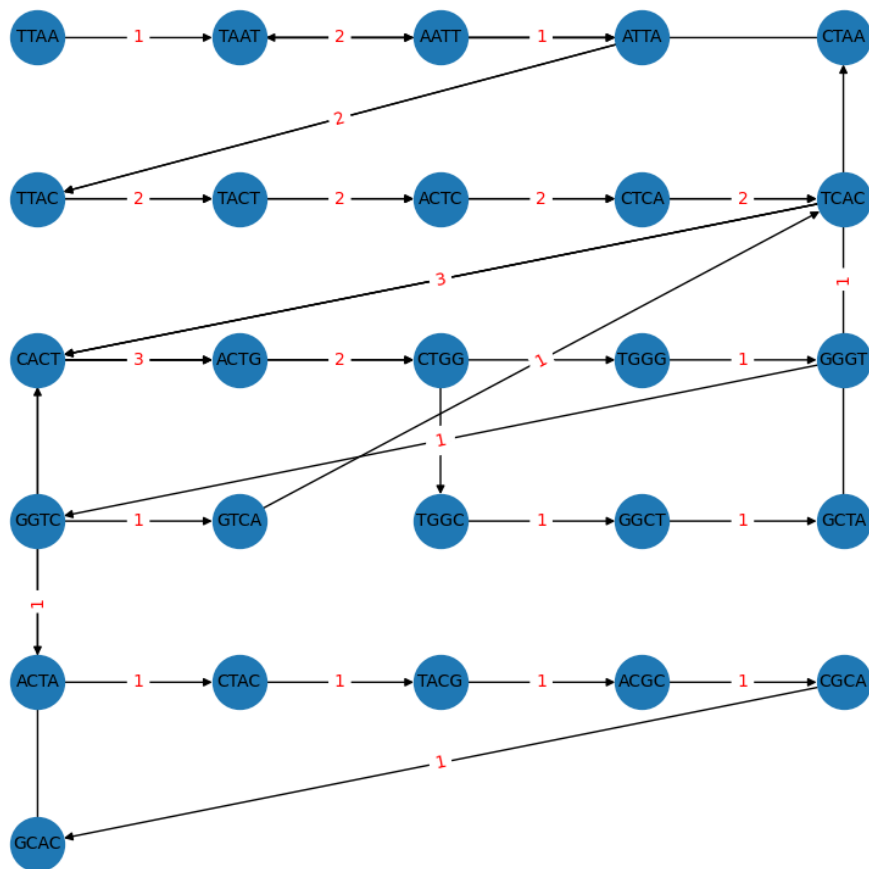
'ACTA', 'CTAC', 'CTAC', 'TACG', 'TACG', 'ACGC', 'ACGC', 'CGCA', 'CGCA', 'GCAC', 'GCAC', 'CACT', 'CACT', 'ACTG']

And the output of `construct_graph(JSON_sequences, k)` is a graph object :

MultiDiGraph with 26 nodes and 40 edges

5. Plot the de Bruijn graph.

The plot of the graph is constructed in the previous step is saved in PNG file name DNA_2



In this graph to deal with multiple edges, the number of repeated edges between nodes are plotted as labels and these values are calculated with the `count_parallel_edges(graph)` function which returns a dictionary as followed:

```
{('TAA', 'TAAT'): 1, ('TAAT', 'AATT'): 2, ('AATT', 'ATTA'): 2, ('ATTA', 'TTAC'): 2, ('CTAA', 'TAAT'): 1, ('TTAC', 'TACT'): 2, ('TACT', 'ACTC'): 2, ('ACTC', 'CTCA'): 2, ('CTCA', 'TCAC'):
```

2, ('TCAC', 'CACT'): 3, ('CACT', 'ACTG'): 3, ('CACT', 'ACTA'): 1, ('ACTG', 'CTGG'): 2, ('CTGG', 'TGGG'): 1, ('CTGG', 'TGGC'): 1, ('TGGG', 'GGGT'): 1, ('GGGT', 'GGTC'): 1, ('GGTC', 'GTCA'): 1, ('GTCA', 'TCAC'): 1, ('TGGC', 'GGCT'): 1, ('GGCT', 'GCTA'): 1, ('GCTA', 'CTAA'): 1, ('ACTA', 'CTAC'): 1, ('CTAC', 'TACG'): 1, ('TACG', 'ACGC'): 1, ('ACGC', 'CGCA'): 1, ('CGCA', 'GCAC'): 1, ('GCAC', 'CACT'): 1}

6. Check whether the de Bruijn graph can be sequenced.

This test is confirmed by the *is_valid_graph(graph)* function, it first checks if the graph is connected, using the *check_connected(graph)*, then it checks for other conditions.

Both these functions return True for DNA_2_5

7. Construct DNA sequence.

After checking if the graph is valid, this function first finds the starting node using the *startnode_eulerian_path(graph)* function. The output of this function is:

TTAA

Then it calls *find_eulerian_path(graph, start_node)* function:

within this function, first the out edges of each node will be saved in a dictionary by the *out_edges_multidigraph(graph)* function. The initial out_edges information for each node is as followed:

```
{'TTAA': ['TAAT'], 'TAAT': ['AATT', 'AATT'], 'AATT': ['ATTA', 'ATTA'], 'ATTA': ['TTAC', 'TTAC'], 'CTAA': ['TAAT'], 'TTAC': ['TACT', 'TACT'], 'TACT': ['ACTC', 'ACTC'], 'ACTC': ['CTCA', 'CTCA'], 'CTCA': ['TCAC', 'TCAC'], 'TCAC': ['CACT', 'CACT', 'CACT'], 'CACT': ['ACTG', 'ACTG', 'ACTG', 'ACTA'], 'ACTG': ['CTGG', 'CTGG'], 'CTGG': ['TGGG', 'TGGC'], 'TGGG': ['GGGT'], 'GGGT': ['GGTC'], 'GGTC': ['GTCA'], 'GTCA': ['TCAC'], 'TGGC': ['GGCT'], 'GGCT': ['GCTA'], 'GCTA': ['CTAA'], 'ACTA': ['CTAC'], 'CTAC': ['TACG'], 'TACG': ['ACGC'], 'ACGC': ['CGCA'], 'CGCA': ['GCAC'], 'GCAC': ['CACT']}
```

Then using the *path_to_stuck(start_node, out_edges)* function, the initial path in the graph (as a list) until it reach a node with no out edge is driven and the dictionary contains the out edges is updated:

Initial path:

```
['TTAA', 'TAAT', 'AATT', 'ATTA', 'TTAC', 'TACT', 'ACTC', 'CTCA', 'TCAC', 'CACT', 'ACTG', 'CTGG', 'TGGG', 'GGGT', 'GGTC', 'GTCA', 'TCAC', 'CACT', 'ACTG', 'CTGG', 'TGGC', 'GGCT',
```

'GCTA', 'CTAA', 'TAAT', 'AATT', 'ATTA', 'TTAC', 'TACT', 'ACTC', 'CTCA', 'TCAC', 'CACT', 'ACTG']

Updated out_edges:

```
{'TTAA': [], 'TAAT': [], 'AATT': [], 'ATTA': [], 'CTAA': [], 'TTAC': [], 'TACT': [], 'ACTC': [], 'CTCA': [], 'TCAC': [], 'CACT': ['ACTA'], 'ACTG': [], 'CTGG': [], 'TGGG': [], 'GGGT': [], 'GGTC': [], 'GTCA': [], 'TGGC': [], 'GGCT': [], 'GCTA': [], 'ACTA': ['CTAC'], 'CTAC': ['TACG'], 'TACG': ['ACGC'], 'ACGC': ['CGCA'], 'CGCA': ['GCAC'], 'GCAC': ['CACT']}
```

The edges that are taken for the path are removed from the dictionary. Now, within the `extend_euler_path(graph, euler_path, out_edges):` function, the node which still has non used edge is selected, in this case CACT is selected and again this node plus the updated dictionary are applied to the `path_to_stuck(start_node, out_edges)` function and the result is:

['CACT', 'ACTA', 'CTAC', 'TACG', 'ACGC', 'CGCA', 'GCAC', 'CACT']

```
{'TTAA': [], 'TAAT': [], 'AATT': [], 'ATTA': [], 'CTAA': [], 'TTAC': [], 'TACT': [], 'ACTC': [], 'CTCA': [], 'TCAC': [], 'CACT': [], 'ACTG': [], 'CTGG': [], 'TGGG': [], 'GGGT': [], 'GGTC': [], 'GTCA': [], 'TGGC': [], 'GGCT': [], 'GCTA': [], 'ACTA': [], 'CTAC': [], 'TACG': [], 'ACGC': [], 'CGCA': [], 'GCAC': []}
```

This path will be replaced by the node CACT in the initial path and since there is no other non visited edge, it is the final Euler path.

['TTAA', 'TAAT', 'AATT', 'ATTA', 'TTAC', 'TACT', 'ACTC', 'CTCA', 'TCAC', 'CACT', 'ACTA', 'CTAC', 'TACG', 'ACGC', 'CGCA', 'GCAC', 'CACT', 'ACTG', 'CTGG', 'TGGG', 'GGGT', 'GGTC', 'GTCA', 'TCAC', 'CACT', 'ACTG', 'CTGG', 'TGGC', 'GGCT', 'GCTA', 'CTAA', 'TAAT', 'AATT', 'ATTA', 'TTAC', 'TACT', 'ACTC', 'CTCA', 'TCAC', 'CACT', 'ACTG']

And finally the DNA sequence correspond this path is
TTAATTACTCACTACGCACTGGGTCACTGGCTAATTACTCACTG

Which is saved in a txt file named DNA_2